

title: Backend Requirements Document — Volunteer Platform

Backend Requirements Document

Volunteer Organization Platform — Frontend → Laravel API Handoff

Source of truth: static analysis of the React/Vite frontend codebase only (`c:\Users\hp\Desktop\volunteer-organization`). Every requirement below is derived from actual frontend code — services, hooks, validation schemas, constants, and components — not assumptions. File citations (`path:line`) are given for load-bearing facts so the backend team can verify against source.

Legend used throughout:

Tag	Meaning
✔ Confirmed built	Inline code comments confirm this already exists and works on the real Laravel backend
🚧 Not implemented	Inline code comments confirm the backend controller/route/column does not exist yet
📄 New requirement	Needed by the frontend; no evidence either way — assume it must be built
❓ Unconfirmed path	Frontend calls this path defensively but marks it TODO / assumed convention
🧮 Computed, not stored	Frontend expects this as a derived value; must not be a manually-written column

1. Tech Stack Context

React 19 + Vite 8, React Router v7 (data router), TanStack React Query v5, React Hook Form + Zod v4, Axios, Tailwind v4. No Redux — global state is a single `AuthContext` (session only) plus React Query for all server data. (`package.json`)

The app runs in two modes controlled by one env var: `VITE_API_MODE=real` hits the live Laravel API; anything else (including unset) uses an in-memory/localStorage mock layer that mirrors the intended real contract (`src/services/api/mockMode.js:12-14`). Every service function branches on this flag, so **the mock code is effectively executable documentation of the intended API contract** — it is the primary source for the request/response shapes in this document.

2. Global API Conventions

All requests go through one Axios instance (`src/services/api/client.js`):

- **Base URL:** `VITE_API_BASE_URL` env var (`client.js:5`).
- **Auth header:** every request reads `localStorage['auth_session']`, parses it, and if `.token` is a non-empty string, sets `Authorization: Bearer {token}` (`client.js:13-27`). The frontend never manages the token in memory/state — it re-reads storage per request.
- **Response envelope:** if a response body is a non-array object with a `data` key (Laravel API Resource shape `{data, meta?, links?}`), the client unwraps it — `response.data` becomes the inner `data`, and `meta` / `links` are hoisted onto the response (`client.js:29-45`). **Every endpoint should either return a bare JSON body, or this Resource envelope — not something else.**
- **401 handling:** any `401` response clears the stored session and hard-redirects to `/login` (loop-guarded) (`client.js:50-64`). No refresh-token flow exists anywhere in the codebase.

- **Error contract expected:**
 - 422 validation errors: `{ message, errors: { field: [msg, ...] } }` — standard Laravel form-request shape (`client.js:80-94` , `getApiFieldErrors`).
 - All other errors: a top-level `message` or `error` string (`client.js:66-74` , `getApiErrorMessage`).
- **Service-layer return shape** (frontend-internal, not literally what Laravel must return): every service function normalizes both mock and real results into `{ success: boolean, data?, error?, fieldErrors? }` before handing it to a hook/component. This means the backend does **not** need to literally return `success: true` — the frontend derives it from HTTP status/axios success — but it does mean **every mutation's success response should include the created/updated resource in the body** so the frontend can populate `data` .
- **File uploads / updates:** any endpoint that both accepts a file **and** is a resource update must accept `POST` with a spoofed method — `FormData.append('_method', 'PUT')` — because PHP cannot parse `$_FILES` on a true `PUT` multipart request (`src/services/api/mediaUpload.js:29-41` , confirmed pattern reused in `opportunities.js` , `volunteer.js` , `organization.js`).

3. Entities & Relationships

Field lists below are the **union of every field the frontend reads or writes**, reconstructed from mock stores, Zod validation schemas, and service payload builders. camelCase = frontend-side name; where the real API uses snake_case, both are shown.

3.1 User (base) — `users` table

Field	Type	Notes
<code>id</code>	string/number	
<code>email</code>	string	unique, login identifier
<code>password</code>	string	hashed server-side; never returned to client (<code>sanitizeUser</code> , <code>src/services/auth.js:29-34</code>)
<code>phone</code> / <code>phone_number</code>	string	required at registration for both account types
<code>roles</code>	string[]	Spatie-style; must be present on login/register response for role resolution (<code>src/services/auth.js:81-98</code>)

3.2 Volunteer profile (extends User)

Field	Type	Notes
<code>firstName / first_name</code>	string	required at registration
<code>lastName / last_name</code>	string	required at registration
<code>educationLevel / education_level</code>	enum	<code>No Formal Education, High School, Diploma, Bachelor's Degree, Master's Degree, PhD</code> (<code>src/utils/auth/VolunteerProfileValidation.js:18-25</code>)
<code>dateOfBirth / birth_date</code>	date	must yield age ≥ 18 (<code>VolunteerProfileValidation.js:53-59</code>)
<code>gender / gendre</code>	enum	schema only allows <code>Female, Male</code> (backend validates lowercase <code>in:male,female</code> per code comment — note the misspelled <code>gendre</code> param name, <code>src/services/volunteer.js:34-35</code>)
<code>city</code>	enum	one of 14 Syrian governorates
<code>skillIds / skill_ids</code>	string[]	≥ 1 required
<code>interests</code>	string	optional, free text
<code>about</code>	string	optional, free text
<code>photo</code>	file	optional, media collection <code>profile_photo</code>

3.3 Organization — `organizations` table

Field	Type	Notes
<code>id</code>	string/number	
<code>name</code> / <code>organization_name</code>	string	2–255 chars
<code>email</code>	string	🚧 derived from <code>owner.email</code> , not a column on <code>organizations</code> (<code>src/utils/api/apiResponseSchemas.js:73-115</code>)
<code>contactPerson</code> / <code>contact_person</code>	string	required
<code>description</code>	string	≥ 10 chars
<code>city</code>	enum	one of 14 governorates
<code>website</code>	string	optional, must be valid URL if present
<code>phone</code>	string	🚧 exists on <code>users.phone_number</code> but <code>OrganizationResource</code> never returns it — always <code>undefined</code> today (<code>src/services/organizations.js:24-28</code>)
<code>imageUrl</code> / <code>profile_image</code>	file URL	logo
<code>verificationDocumentUrl</code> / <code>verification_document</code>	file URL	uploaded once at registration
<code>status</code>	enum	🚧 column does not exist on <code>organizations</code> table yet — API always returns <code>null</code> , frontend fakes <code>'pending'</code> as a stopgap (<code>apiResponseSchemas.js:70-72,83,109,160</code>)
<code>rejectionReason</code>	string null	🚧 concept doesn't exist in backend yet — always <code>null</code> in real mode (<code>apiResponseSchemas.js:110-113</code>)
<code>reviewedAt</code>	datetime	admin decision timestamp
<code>requestedAt</code> / <code>created_at</code>	datetime	
<code>owner</code>	<code>{id, name, email}</code> null	

3.4 Opportunity — opportunities table

Field	Type	Notes
id	string/number	
title	string	3–255 chars
description	string	≥ 20 chars
status	enum	 computed , see §7 — registration_open registration_closed in_progress completed
registrationClosedManually	boolean	the only manually-writable status-adjacent field (org toggle)
startDate / start_date	date	
endDate / end_date	date	must be ≥ startDate
registerStartAt	date	
registerEndAt	date	must be ≥ registerStartAt , ≤ startDate
location / city	string	governorate
minHours , maxHours	number	maxHours ≥ minHours , both ≥ 1
totalHours	number	≥ 1
minVolunteers , maxVolunteers	integer	maxVolunteers ≥ minVolunteers , both ≥ 1
currentVolunteers / current_volu	integer	 computed live = count of participations with status = accepted ; must not be a manually incremented/decremented counter (src/services/opportunities.js:42-60)
isGroup	boolean	default false ; feeds an achievement rule (§13)
category	{id, name}	FK
skills	[{id, name}]	many-to-many
organization	{id, name, imageUrl, phone, status}	nested
image	file URL	cover image
createdAt	datetime	needed for dashboard analytics bucketing (§15)

Removed field (explicit backend decision): minAge / maxAge — age restriction on opportunities was fully removed per backend team request (src/utils/opportunityValidation.js:4-5).

3.5 Participation — `opportunity_volunteer` pivot table

Field	Type	Notes
<code>id</code>	string/number	
<code>volunteerId</code> , <code>opportunityId</code>	FK	
<code>status</code>	enum	stored: <code>pending</code> <code>accepted</code> <code>rejected</code> <code>withdrawn</code> . <code>expired</code> is  display-only, never stored (§8)
<code>committedHours</code>	number	set at apply time, bounded to <code>[opportunity.minHours, opportunity.maxHours]</code>
<code>hoursLogged</code>	number null	 column does not exist yet — new nullable column required (<code>src/services/participations.js:216-222</code>)
<code>joinedDate</code> / <code>participatedAt</code>	date	
<code>rejectionReason</code>	string	only set when <code>status = rejected</code>
<code>withdrawnDate</code>	date	only set when <code>status = withdrawn</code>
<code>canWithdraw</code>	boolean	optional explicit server override; if present the frontend trusts it over local computation (<code>src/utils/participationPolicy.js:27</code>)

3.6 Category — `categories` table

`id`, `name` (unique), `description`, `opportunitiesCount` ( computed via `withCount('opportunities')`).  Confirmed built already (`src/services/categories.js:56-61`).

3.7 Skill — `skills` table

`id`, `name` (unique), `categoryId`, category object **embedded** in responses (not just FK id — see §18).  **SkillController is entirely unimplemented** and the `skills` table currently only has `id` + timestamps (`src/services/skills.js:96-104`).

3.8 Governorate/City

`id`, `nameEn` (required, unique), `nameAr` (required in data model, but no frontend form field collects it – see §23), `slug` (auto-derived from `nameEn`) . Currently 100% static frontend data in 4 duplicated locations (§23) — no live GET endpoint wired anywhere yet.

3.9 Achievement — platform-wide, not org-scoped

`id`, `name`, `description`, `unlocked` (boolean), `earnedDate` (date|null), `progress: {current, target}|null` . Rules must be evaluated **entirely server-side** (§13).

3.10 Notification — not built at all

`id`, `type`, `title`, `description`, `href`, `seen` (boolean) .

Entity-relationship summary

```

User —1:1— Volunteer profile
User —1:1— Organization (owner)
Organization —1:N— Opportunity
Opportunity —N:1— Category
Opportunity —N:N— Skill
Category —1:N— Skill
Volunteer —N:N— Opportunity (through Participation / opportunity_volunteer)
Volunteer —N:N— Achievement (platform-wide unlock state)

```

4. User Roles & Permissions

Three account types ([src/constants/auth/accountTypes.js:1-7](#)): **volunteer** , **organization** , **admin** . Admin has **no public registration screen** — it must be pre-assigned via a Spatie role directly in the backend (comment, [accountTypes.js:4-6](#)). Role resolution priority when a user carries multiple roles: **admin > organization > volunteer** ([src/services/auth.js:88-90](#)).

Capability	Public/Guest	Volunteer	Organization	Admin
Browse opportunities & organizations (public listing/details)	✓	✓	✓	✓
View/edit own volunteer profile	—	✓	—	—
Apply / withdraw from opportunities	—	✓	—	—
View own participation history & hours summary	—	✓	—	—
View achievements	—	✓	—	—
View/edit own organization profile	—	—	✓	—
Resubmit verification document	—	—	✓ (only when rejected)	—
Create / edit / delete own opportunities	—	—	✓	—
Toggle own opportunity registration open/closed	—	—	✓ (only if verified)	—
View/accept/reject applicants, log final hours	—	—	✓ (only if verified)	—
View organization dashboard/analytics	—	—	✓	—
CRUD categories, skills, governorates	—	—	—	✓
Review/verify/reject/suspend organizations	—	—	—	✓
View platform-wide stats dashboard	—	—	—	✓
Manage own admin profile/password	—	—	—	✓

Route-level enforcement is via `ProtectedRoute` ([src/app/routes/ProtectedRoute.jsx:5-18](#)): unauthenticated → redirect [/login](#) ; authenticated but wrong role → redirect [/](#) (home), **not** a 403 page. **The backend must independently re-enforce every one of these boundaries — the frontend guard is UI-only.**

5. Authentication & Authorization

5.1 Token

- Bearer token, attached as `Authorization: Bearer {token}` on every request (§2).

- Stored client-side in `localStorage['auth_session']` as `{ user, token, accountType }` — no `isAuthenticated` flag persisted (derived at load).
- **Undetermined (needs backend decision, \$24):** token lifetime/expiry and refresh strategy are not modeled anywhere in the frontend — assume Sanctum personal access tokens with no client-side refresh flow unless backend specifies otherwise.
- **Logout is 100% client-side** — clears storage + React Query cache; **no server call is made** (`src/context/AuthContext.jsx:70-76`). If the backend wants server-side token revocation, a logout endpoint is a new requirement to raise with the frontend team (not currently called).

5.2 Register — `POST /register`

Always sent as **multipart FormData**, even without a file (because organization registration includes one)

(`src/services/auth.js:121-146,179`).

Field (snake_case sent)	Volunteer	Organization
<code>account_type</code>	✓	✓
<code>email</code> , <code>password</code> , <code>password_confirmation</code> (= password)	✓	✓
<code>phone_number</code>	✓	✓
<code>first_name</code> , <code>last_name</code>	✓	—
<code>organization_name</code> , <code>contact_person</code>	—	✓
<code>verification_document</code> (file, required)	—	✓

Response must be `{ user: {id, email, ...}, token: non-empty string }`, ideally with `roles[]` on the user object.

Validated by `authResponseSchema` — mismatch produces a generic error, not a crash

(`src/utils/api/apiResponseSchemas.js:23-57`).

5.3 Login — `POST /login`

Plain JSON `{ email, password }`. Same response contract as register.

5.4 Password reset — ? paths unconfirmed (Fortify/Breeze convention assumed)

- `POST /forgot-password { email }` — **must always return generic success** regardless of whether the email exists (anti-enumeration; explicit frontend requirement, `src/services/auth.js:320-323`).
- `POST /reset-password { token, email, password, password_confirmation }` — token should be single-use, ~1 hour TTL (mirrors the mock implementation exactly, `src/services/mock/mockPasswordResetStore.js:5`).

5.5 Admin self-service — ? paths unconfirmed

- `PUT /admin/profile { name, email, phone, bio }`
- `PUT /admin/password { currentPassword, newPassword }`

5.6 Field-error mapping (Laravel snake_case → form field)

`phone_number→phone`, `first_name→firstName`, `last_name→lastName`, `organization_name→orgName`, `contact_person→contactPerson`, `verification_document→verificationImage`, `password_confirmation→confirmPassword` (`src/services/auth.js:47-57`). Any 422 the backend returns on these fields will surface correctly on the matching form input — using a different field name will silently fail to bind.

5.7 Volunteer profile-completion gate

Applies **globally**, to every route inside the main layout (public pages included), for authenticated volunteers only. If ALL of `educationLevel`, `dateOfBirth`, `gender`, `city`, `skillIds.length ≥ 1` aren't set, the volunteer is force-redirected to their profile page and cannot navigate anywhere else, including Home (`src/app/routes/RequireCompleteProfile.jsx`, `src/utils/auth/profileCompletion.js:18-29`). This is enforced client-side only today — flagged in §24 as worth backend-side enforcement too if the profile-edit endpoint is hit directly (e.g. via API tooling bypassing the SPA).

6. Route Map (frontend reference)

Path	Access	Page
<code>/</code> <code>/about</code> <code>/participates</code> <code>/opportunities</code> <code>/opportunities/:id</code> <code>/organizations</code> <code>/organizations/:id</code>	Public	Home, About, Success stories, listings, details
<code>/volunteer-profile</code> <code>/explore</code> <code>/my-volunteering</code>	Volunteer only	Profile, opportunities (aliased), participations
<code>/organization-profile</code> <code>/dashboard</code> <code>/my-causes</code> <code>/my-causes/create</code> <code>/my-causes/:id/edit</code> <code>/applicants/:id</code>	Organization only	Profile, dashboard, opportunity CRUD, applicants
<code>/admin/dashboard</code> <code>/admin/organizations</code> <code>/admin/categories</code> <code>/admin/cities</code> <code>/admin/profile</code> <code>/admin/settings</code>	Admin only	Platform stats, org review, catalog CRUD, profile
<code>/login</code> <code>/register</code> <code>/forgot-password</code> <code>/reset-password</code>	Guest (outside layout)	Auth flows

(`src/App.jsx:56-130`)

7. Opportunity Lifecycle

7.1 Status values (`src/constants/opportunityStatus.js:14-19`)

`registration_open`, `registration_closed`, `in_progress`, `completed` — this replaced an older 2-state model per an explicit prior backend requirement (comment, same file lines 1-19).

7.2 Computation rule — must be computed server-side, not stored

(`src/utils/opportunityStatus.js:16-39`)

```
if now > end_date:                → completed
elif now >= start_date:           → in_progress
elif current_volu >= max_volu
    OR now > register_end_at
    OR registration_closed_manually == true:  → registration_closed
else:                             → registration_open
```

`current_volu` itself must be a **live count of `accepted` participations**, never a manually maintained counter (§3.4).

7.3 Transitions

From → To	Trigger	Actor	Notes
(create) → <code>registration_open</code> / <code>closed</code>	New opportunity	Organization	<code>POST /opportunities</code> (multipart)
<code>registration_open</code> ↔ <code>registration_closed</code>	Manual toggle	Organization	<code>PATCH /opportunities/{id}/status {status}</code> — button only enabled pre- <code>in_progress</code> , and only for verified orgs
any → <code>registration_closed</code>	Auto: full, or registration window passed	System	computed, no write
any → <code>in_progress</code>	Auto: <code>now ≥ start_date</code>	System	computed, no write
any → <code>completed</code>	Auto: <code>now > end_date</code>	System	computed, no write
(any) → (deleted)	Organization deletes	Organization	<code>DELETE /opportunities/{id}</code>
(any) → (edited)	Organization edits	Organization	<code>POST /opportunities/{id}</code> + <code>_method=PUT</code> , multipart

`isSuccessfulOpportunity` = `status == completed AND current_volu ≥ min_volu` — used to decide "Success Story" eligibility (§17.4); this is **stricter** than `status=completed` alone, and the real-mode success-stories call currently only filters by `status=completed` — flagged as a contract gap in §23.

8. Participation (Application) Lifecycle

8.1 Stored status values (`src/constants/participationStatus.js:19-25`)

`pending` , `accepted` , `rejected` , `withdrawn` — confirmed as the only 4 values actually persisted (comment, lines 3-10).

8.2 Computed-only values (never written to DB)

- `expired` — display substitute when `status = pending` **and** `now ≥ opportunity.start_date` (org never decided in time) (`participationStatus.js:54-61`).
- `active` / `completed` (display projection) — when `status = accepted` , shown as `active` if the opportunity is `in_progress` , or `completed` if the opportunity is `completed` ; the stored value never changes from `accepted` (`src/utils/participationDisplayStatus.js:28-37`).

8.3 Full transition table

From	To	Trigger	Actor	Guard
(none)	<code>pending</code>	Apply	Volunteer	Opportunity must be <code>registration_open</code> ; <code>committedHours</code> within <code>[minHours, maxHours]</code> ; no existing active (<code>pending</code> / <code>accepted</code>) participation by this volunteer for this opportunity
<code>pending</code>	<code>accepted</code>	Decision	Organization	Org must be verified; final, irreversible , no confirmation dialog in UI
<code>pending</code>	<code>rejected</code>	Decision	Organization	Org must be verified; reason is mandatory ; final, irreversible
<code>pending</code> / <code>accepted</code>	<code>withdrawn</code>	Withdraw	Volunteer	<code>canWithdraw</code> = opportunity registration still <code>open</code> (NOT based on start date — see below)
<code>rejected</code> / <code>withdrawn</code>	<code>pending</code> (new application)	Re-apply	Volunteer	Explicitly allowed — these are terminal-but-not-blocking states (<code>src/services/opportunities.js:578-581</code>)

Row is never deleted on withdrawal — this was an explicit design change: withdrawal used to hard-delete the pivot row, which silently erased the applicant from the org's view with no explanation; now it's kept as `status = withdrawn` , distinct from `rejected` , so both sides retain a transparent history (`src/constants/participationStatus.js:12-18`).

`canWithdraw` **guard precisely** (`src/utils/participationPolicy.js:26-33`): the deciding factor is the **opportunity's registration status**, not its start date — once registration closes (full, window passed, or manually closed by the org), withdrawal is blocked immediately even if the opportunity itself is weeks from starting. If the backend returns an explicit `canWithdraw` boolean on the participation object, the frontend trusts it directly over recomputing.

8.4 Endpoints

- `POST /opportunities/{id}/participate { committed_hours }`
- `PUT /participations/{id} { status, rejection_reason? }` — accept/reject
- `PUT /participations/{id} { status: 'withdrawn' }` — withdraw
- `PATCH /participations/{id}/hours { hours }` —  **not built**; requires new `hours_logged` column (\$3.5, \$9)

9. Hours Management

Two distinct numbers per participation, never conflated:

Field	Set by	When	Bound
<code>committedHours</code>	Volunteer	At apply time	<code>[opportunity.minHours, opportunity.maxHours]</code> inclusive
<code>hoursLogged</code>	Organization	After opportunity ends, only for <code>accepted</code> participations	≥ 0 , no upper bound (org can log more or less than committed)

- Commit-time validation message expected server-side too: "Please commit to a number between {min} and {max} hours." (mirrors mock, `src/services/opportunities.js:553-559`).
- Org's "Manage/Edit hours" action is only reachable once `status = accepted` **and** `opportunity.endDate < now` (`src/components/organization/ApplicantCard.jsx:37` ; `src/pages/applicantsList.jsx:60-62`).
- No decimal-precision or step constraint found anywhere — plain finite-number validation only.

- 🚧 **Backend gap:** this whole feature needs a new nullable `hours_logged` column on the pivot table plus the `PATCH /participations/{id}/hours` endpoint — explicitly flagged as unbuilt in frontend comments (`src/services/participations.js:216-222`).

10. Volunteer Requirements

- Registration fields: \$3.2 base + \$5.2 table.
- Profile-completion required fields (gates entire app, \$5.7): `educationLevel`, `dateOfBirth`, `gender`, `city`, `skillIds (≥1)`. `interests` / `about` optional.
- ⚠️ **Schema/UI mismatch to flag:** the Zod schema allows 6 education levels, but the actual profile form only renders 4 (Master's/PhD are commented out) — backend should accept all 6 regardless, this is a frontend gap, not a backend one (`src/components/volunteerProfile/ProfileForm.jsx:16-23` vs `VolunteerProfileValidation.js:18-25`).
- ⚠️ **Gender mismatch:** validation schema only allows `Female` / `Male`, but a `GenderBadge` component elsewhere renders a third value `"Prefer not to say"` — backend should clarify whether a third gender option is intended (§24).
- Volunteers can: apply/withdraw, log a committed-hours pledge, view own participation list + effective/display status, view a personal hours summary (aggregated from participations), view achievements, receive notifications (achievement unlocked, hours confirmed, application accepted/rejected, upcoming-opportunity reminder 2 days before start).

11. Organization Requirements & Verification Lifecycle

11.1 Status values (`src/constants/organizationStatus.js:2-7`)

`pending` (default) → `verified` | `rejected` | `suspended` (admin-only transitions, §12).

11.2 Effects of each status

- `pending`: awaiting admin review, banner shown, opportunity actions likely still available in current mock but production intent should be confirmed (§24).
- `verified`: full access — can toggle opportunity status, accept/reject applicants, log hours.
- `rejected`: **account fully locked** — cannot post opportunities, review applicants, or use org services until a **new verification document is resubmitted**, which resets status back to `pending` server-side (`src/components/OrgProfile/RejectedVerificationPanel.jsx:50-53` ; `src/services/organization.js:196-200`).
- `suspended`: admin-only action, only available on already- `verified` orgs (`src/components/admin/OrganizationReviewCard.jsx:99-112`); same lockout expectation as rejected.

11.3 Endpoints

- `GET /organizations/{id}` / `PUT /organizations/{id}` {`name`, `description`, `city`, `website`, `contact_person`} — profile (no `email` field — not a column on `organizations`).
- 🚧 **New requirement:** `POST /organizations/{id}/verification-document` (multipart, field `verification_document`) — resubmission after rejection; must reset `status→pending` and clear `rejection_reason` server-side.
- 🚧 **Confirmed gap:** `OrganizationController::update` today only does `$organization->update($request->validated())` — it does **not** handle any file/logo upload, so there is currently no way to update an org's logo after registration (`src/services/organization.js:1-24`).

12. Admin Requirements

- **Catalog CRUD:** Categories (✅ backend confirmed implemented — `CategoryController` store/update/destroy match `CategoryRequest`, `src/services/categories.js:56-61`), Skills (🚧 `SkillController` entirely unimplemented, table lacks working `categoryId`, `src/services/skills.js:96-104`), Governorates (📌 planned CRUD endpoints exist client-side as calls, but confirmed no live GET endpoint is wired into any form yet — see §23).
- **Organization review:** `GET /admin/organizations?status=pending|verified|rejected|suspended`, `PATCH /admin/organizations/{id}/verify {status, reason?}` — `reason` **required** for `rejected` / `suspended`, not required for `verified`. 🚧 **No admin namespace/routes exist yet at all** — `role:admin` Spatie middleware is planned but not built (`src/services/admin.js:1-13`).
- **Own profile/password:** `PUT /admin/profile`, `PUT /admin/password` (🔗 paths unconfirmed).
- **Platform stats:** `GET /stats/summary` → `{volunteersCount, organizationsCount, opportunitiesCount}`. A 4th field `citiesCoveredCount` is referenced in the UI but explicitly flagged **unconfirmed** — ask backend whether it exists or drop it from the dashboard (`src/services/stats.js:5-7`).
- **Admin dashboard** is currently composed **client-side** from 3 separate queries (stats + categories + full org list) — no dedicated endpoint exists. A `queryKeys.admin.dashboard` cache key is already defined in the frontend but **no hook uses it** — this is either a signal of a planned-but-unbuilt `GET /admin/dashboard` aggregate endpoint, or dead code to clean up (flag with frontend team, §24).

13. Achievements System

Three definitions, all evaluated against a volunteer's participation history (`src/services/achievements.js:36-52,92-94`):

ID	Name	Unlock rule
a1	First Volunteering Opportunity	<code>completedOpportunitiesCount ≥ 1</code>
a2	10 Volunteer Hours	<code>totalConfirmedHours ≥ 10</code>
a3	Completion of Three Group Activities	<code>completedOpportunitiesCount ≥ 3</code>

Explicit, repeated instruction in the code: achievement unlocking must be **entirely backend business logic**, not admin-set and not frontend-decided — "Laravel should evaluate these automatically (e.g., via an Eloquent model event/observer triggered when a participation completes) and flip `unlocked=true` + set `earnedDate`" (`achievements.js:1-20`). The frontend only ever renders whatever `GET /volunteers/{id|me}/achievements` returns: `[{id, name, description, unlocked, earnedDate, progress: {current,target}|null}]`.

Achievements are **platform-wide**, not per-organization (`achievements.js:56-57`) — worth reconfirming as a deliberate product decision (§24).

⚠️ **Ambiguity to flag:** achievement `a3`'s name says "group activities," but the mock rule that computes it just counts *any* 3 completed opportunities — it does not filter by the opportunity's `isGroup` flag. Confirm with product/backend whether `a3` should specifically require `isGroup = true` opportunities.

14. Notifications

🚧 **No real endpoint exists yet** ("approved by backend team but not built" per code comment, `src/services/notifications.js:1-13`). Intended shape once built: `GET /notifications` → `[{id, type, title, description, href, seen}]`.

Types the frontend already renders (derived client-side today): `achievement` , `hours` , `status-accepted` , `status-rejected` , `opportunity-reminder` , `org-verified` , `org-rejected` , `applicant-withdrawn` .

⚠ **Gap:** "seen/read" state is currently tracked **entirely client-side** via 5 separate `localStorage` maps, with no server representation at all. If/when this endpoint is built, a `PATCH /notifications/{id}/read` -style mechanism will be needed, replacing the `localStorage` trackers.

`useRecentUpdates` polls this data source **every 5 seconds** for authenticated volunteers and for organizations that have an `organizationId` (`src/hooks/useRecentUpdates.js:23,31-34`) — whatever endpoint backs this must be cheap enough to poll per active user.

15. Dashboards & Analytics (Organization)

No dedicated backend endpoint exists yet — currently composed client-side from `fetchMyOpportunities()` + per-opportunity `fetchApplicantsForOpportunity()` calls. Intended future single endpoint: `GET /organizations/me/dashboard` (`src/services/dashboard.js:1-9`).

Expected response shape:

```
{
  totalOpportunities, openOpportunities, totalVolunteers, pendingRequests,
  completionRate,           // round(completed / total * 100)
  totalHoursPledged,        // Σ committedHours across all NON-REJECTED applicants
  opportunitiesBreakdown: [{id, title, currentVolunteers, maxVolunteers}],
  recentActivity: [{id, volunteerName, opportunityTitle, opportunityId, status, date}], // top 5 by date desc
  analyticsTrends: {
    metrics: [{key, label}],
    weekly: [ 7 daily buckets ],
    monthly: [ 6 monthly buckets ]
  }
}
```

Each trend bucket: `{key, label, opportunitiesCreated, applicationsReceived, acceptedVolunteers, totalVolunteerHours}` . Requires opportunities to expose a `created_at` -equivalent timestamp for correct bucketing (`src/services/dashboard.js:80-85`).

⚠ **Two "hours" numbers are intentionally different — both must be exposed distinctly:** `totalHoursPledged` sums `committedHours` for all **non-rejected** applicants; `analyticsTrends.totalVolunteerHours` sums `hoursLogged ?? committedHours` only for **accepted** applicants (`src/services/dashboard.js:134-137,265-271`).

16. File Uploads

Constraint	Value
Allowed MIME types	<code>image/jpeg</code> , <code>image/png</code> , <code>image/webp</code> (<code>src/constants/media.js:4</code>)
Max size	2 MB (<code>media.js:7</code>)
Upload mechanism	<code>FormData</code> ; updates use <code>POST</code> + <code>_method=PUT</code>

Use case	Field name	Media collection
Volunteer profile photo	photo	profile_photo
Organization logo	photo (generic helper)	profile_image
Organization verification document (registration + resubmission)	verification_document	verification_documents — image only, no PDF support anywhere in the frontend
Opportunity cover image	image	 not yet named in MEDIA_COLLECTIONS — planned, commented out (media.js:13-14)

Backend should mirror the same limits server-side: `image|mimes:jpeg,png,webp|max:2048` .

17. Filtering, Search, Sort, Pagination

17.1 GET /opportunities

Params sent defensively by the frontend: `search`, `categoryId`, `categoryIds[]`, `skillId`, `skillIds[]`, `location` . 

Confirmed gap: `OpportunityController` currently has **no implemented actions at all** — none of these filters exist server-side today.

- The opportunities list page applies one additional **hard client-side filter regardless of backend**: only `status = registration_open` opportunities are ever shown on the main list (other statuses are reachable only via direct links or an org's "past opportunities" view).
- No pagination UI exists on this page — full result set is rendered.

17.2 GET /volunteers/me/suggested-opportunities

Personalization params: `skillIds[]`, `city` . Response is loosely typed on purpose (accepts `matchReason` / `match_reason` / `reason` / `explanation` , or a numeric `probability` / `score`) — backend has flexibility in the exact field name for its match output.

17.3 GET /organizations

-  Confirmed paginated server-side via Laravel `paginate(15)` , envelope nested at `data.data` .
-  **Confirmed gap:** `index()` has **no search filter logic** — frontend sends `?search=` defensively for future support only.
- Frontend also sends `status=verified` but since the `status` column doesn't exist, this has no effect today (frontend already defensively treats `null` status as "pass the filter" rather than excluding everything).

17.4 Applicants list (GET /opportunities/{id}/participants)

Entirely client-side filtering/sort/pagination — the endpoint returns the full applicant list in one call; no query params are sent. Client applies: `search` (name substring), `statusFilter` (`all`|`pending`|`accepted`|`rejected` — note: no `withdrawn` / `expired` filter option even though those statuses exist), `sortOrder` (`newest`|`oldest` by `participatedAt`), and a "show more" (load-more) pagination pattern applied **after** filtering.

17.5 Success stories (GET /opportunities?status=completed)

 Real-mode call only filters by `status=completed` ; it does **not** also require `current_volu ≥ min_volu` the way the mock does — meaning an opportunity that ended with zero volunteers could incorrectly appear as a "success story" today. Recommend the endpoint (or a dedicated success-stories endpoint) apply the stricter filter server-side.

18. React Query / Data Synchronization Requirements

- Global cache defaults: `staleTime: 60s`, `retry: 1`, `refetchOnWindowFocus: false` (`src/app/queryClient.js`).
- Query key hierarchy is resource-rooted (`src/app/queryKeys.js`) — e.g. `opportunities.all` is a prefix that, when invalidated, implicitly invalidates `list/suggested/detail/mine/completed` . **Every opportunity mutation (create/update/delete/toggle-status/participate) is expected to affect all of these views consistently** — i.e. read-after-write consistency on a single `opportunities` resource.
- Mutations widely assume the response body includes the created/updated resource (used to patch the cache directly via `setQueryData`) — e.g. category/skill/city create-mutations expect `result.data` to be the new object; the city update-mutation matches the cache entry by `result.data.id` , implying **the response's `id` must equal the requested resource's `id`**.
-  **Skills responses must embed a full nested `category` object**, not just a `categoryId` — confirmed by frontend comments explaining why skill mutations use full-list invalidation instead of optimistic patching (rebuilding the nested object client-side was judged not worth it) (`useCreateSkillMutation.js` / `useUpdateSkillMutation.js` comments).
-  **Contract inconsistency to resolve:** `useReviewOrganizationMutation` reads `result.status` / `result.reason` at the **top level** of the mutation response, while every other mutation in the codebase reads `result.data {...}` . Recommend standardizing before backend implementation.
- Polling: only one hook polls on an interval — `useRecentUpdates` at 5 seconds (§14). All other data hooks rely on cache invalidation/refetch-on-navigation, not polling. No websockets/SSE anywhere.

19. Full Endpoint Catalog

Method/path as called by the frontend; **Status** column uses the legend at the top of this document.

Auth

Method & Path	Request	Response	Status
POST <code>/register</code>	multipart, see §5.2	<code>{user, token}</code>	
POST <code>/login</code>	<code>{email, password}</code>	<code>{user, token}</code>	
POST <code>/forgot-password</code>	<code>{email}</code>	generic success always	
POST <code>/reset-password</code>	<code>{token, email, password, password_confirmation}</code>	success/error	
PUT <code>/admin/profile</code>	<code>{name, email, phone, bio}</code>	updated admin	
PUT <code>/admin/password</code>	<code>{currentPassword, newPassword}</code>	success/error	

Volunteer

Method & Path	Request	Response	Status
POST <code>/volunteers/{id}</code> + <code>_method=PUT</code>	multipart, §3.2	updated profile	
GET <code>/volunteers/{id me}/achievements</code>	–	<code>Achievement[]</code>	 (rules must be server-computed, §13)
GET <code>/volunteers/me/participations</code>	–	<code>Participation[]</code> (§8.4 shape)	
GET <code>/volunteers/me/suggested-opportunities</code>	<code>skillIds[]</code> , <code>city</code>	<code>Opportunity[]</code> w/ match metadata	

Organization profile & directory

Method & Path	Request	Response	Status
GET /organizations	search, status	paginated list	🚧 search not implemented; status column missing
GET /organizations/{id}	–	organization	✅ (single fetch works)
PUT /organizations/{id}	{name, description, city, website, contact_person}	organization	🚧 no file/logo handling
POST /organizations/{id}/verification-document	multipart verification_document	organization (status→pending)	NEW
GET /organizations/{id}/opportunities	–	Opportunity[]	NEW

Opportunities

Method & Path	Request	Response	Status
GET /opportunities	search, categoryId(s), skillId(s), location, status	paginated/list	🚧 controller has no implemented actions
GET /opportunities/{id}	–	{opportunity, similar}	🚧
GET /organizations/me/opportunities	–	Opportunity[]	NEW
POST /opportunities	multipart, \$3.4	opportunity	🚧
POST /opportunities/{id} + _method=PUT	multipart	opportunity	🚧
DELETE /opportunities/{id}	–	–	🚧
PATCH /opportunities/{id}/status	{status}	opportunity	🚧
POST /opportunities/{id}/participate	{committed_hours}	participation	NEW

Participations

Method & Path	Request	Response	Status
GET /opportunities/{id}/participants	–	applicant list (\$6.2 fields)	NEW
PUT /participations/{id}	{status, rejection_reason?}	participation	NEW
PUT /participations/{id}	{status: 'withdrawn'}	participation	NEW
PATCH /participations/{id}/hours	{hours}	participation	🚧 needs new column + route

Catalog (admin)

Method & Path	Request	Response	Status
GET/POST/PUT/DELETE /categories/{id}	{name, description}	category	✅ confirmed built
GET/POST/PUT/DELETE /skills/{id}	{name, categoryId}	skill (with nested category)	🚧 controller unimplemented
GET/POST/PUT/DELETE /governorates/{id}	{nameAr, nameEn}	governorate	🆕 no GET wired anywhere yet

Admin

Method & Path	Request	Response	Status
GET /admin/organizations	status	paginated list	🚧 no admin namespace exists
PATCH /admin/organizations/{id}/verify	{status, reason?}	organization	🚧
GET /admin/dashboard	–	aggregate stats	🆕 (candidate; currently unused key in frontend)

Misc

Method & Path	Request	Response	Status
GET /stats/summary	–	{volunteersCount, organizationsCount, opportunitiesCount, citiesCoveredCount?}	❓ last field unconfirmed
GET /notifications	–	[[{id, type, title, description, href, seen}]]	🚧 not built

20. Validation Rules Reference

Rule	Value	Source
Email	standard email format	<code>src/utils/auth/validation.js:41</code>
Register/reset password	≥ 8 chars	<code>validation.js:45,28</code>
Login password	non-empty only	<code>validation.js:19</code>
Password confirmation	must match <code>password</code>	<code>validation.js:31-34</code>
Volunteer age	≥ 18 (from <code>dateOfBirth</code>)	<code>VolunteerProfileValidation.js:56-59</code>
Org name	2–255 chars	<code>OrganizationProfileValidation.js:43-47</code>
Org description	≥ 10 chars	<code>OrganizationProfileValidation.js:49-52</code>
Org website	valid URL or empty	<code>OrganizationProfileValidation.js:56-60</code>
Governorate/city	one of 14 fixed values	<code>VolunteerProfileValidation.js:27-42</code>
Education level	one of 6 fixed values	<code>VolunteerProfileValidation.js:18-25</code>
Gender	<code>Female</code> / <code>Male</code> only (see §24 re: 3rd value)	<code>VolunteerProfileValidation.js:16</code>
Skills	array, ≥ 1	<code>VolunteerProfileValidation.js:68</code>
Opportunity title	3–255 chars	<code>opportunityValidation.js:12-15</code>
Opportunity description	≥ 20 chars	<code>opportunityValidation.js:17-20</code>
Opportunity dates	<code>endDate $\geq$ startDate ;</code> <code>registerStartAt $\leq$</code> <code>registerEndAt $\leq$ startDate</code>	<code>opportunityValidation.js:56-75</code>
Opportunity hours	<code>minHours, maxHours,</code> <code>totalHours ≥ 1 ; maxHours $\geq$</code> <code>minHours</code>	<code>opportunityValidation.js:37-40,60-63</code>
Opportunity volunteer caps	<code>minVolunteers, maxVolunteers</code> <code>≥ 1 integers; max $\geq$ min</code>	<code>opportunityValidation.js:42-51,64-67</code>
Committed hours (apply)	within <code>[opportunity.minHours,</code> <code>opportunity.maxHours]</code>	<code>src/services/opportunities.js:553-559</code>
Hours logged (org confirm)	<code>≥ 0</code> , no upper bound	<code>src/components/organization/ManageHoursModal.jsx:30-34</code>
Category/Skill/Governorate name	required, unique (case-insensitive)	<code>categories.js</code> , <code>skills.js</code> , <code>syrianGovernorates.js</code>
Org review reason	required for <code>rejected</code> / <code>suspended</code> , not for <code>verified</code>	<code>src/services/admin.js:117-123</code>
Image upload	JPG/PNG/WEBP, ≤ 2 MB	<code>src/constants/media.js:4-7</code>

No password complexity rules (uppercase/digit/symbol) exist anywhere — length-only.

21. Business Rules the Backend Must Enforce

1. Opportunity status is always derived from dates/counts, never manually set except the `registrationClosedManually` toggle (§7.2).
2. `current_volu` is a live count of `accepted` participations — never a stored increment/decrement counter (§3.4).
3. A volunteer cannot have two simultaneously **active** (`pending` / `accepted`) participations for the same opportunity; re-applying after `rejected` / `withdrawn` is explicitly allowed.
4. Withdrawal is only valid while the opportunity's registration is still open — not gated by the opportunity's start date directly.
5. Rejecting or suspending an organization requires a non-empty `reason`; approving does not.
6. Resubmitting a verification document must reset an organization's status to `pending` and clear its rejection reason server-side.
7. Achievement unlocking must be computed and persisted entirely server-side (event/observer-driven on participation completion) — never admin-set, never frontend-computed in production.
8. Anti-enumeration: `forgot-password` must always return a generic success response regardless of whether the email exists.
9. Password reset tokens should be single-use and time-limited (mock mirrors Laravel's ~1 hour default).
10. File-upload update endpoints must accept `POST` + `_method=PUT` override (PHP cannot parse multipart on true `PUT`).
11. Withdrawn participations must never be hard-deleted — kept as `status = withdrawn`, distinct from `rejected`, to preserve a transparent history for both volunteer and organization.

22. Backend Team Checklist

Blocking / must-build before real-mode cutover:

- ☐ Implement `OpportunityController` — all actions (index with filters, show, store, update, destroy, status toggle, participate) are currently empty.
- ☐ Implement `SkillController` — all actions currently empty; ensure `skills` table has a working `categoryId` column and responses embed a nested `category` object.
- ☐ Add `hours_logged` nullable column to `opportunity_volunteer` pivot + build `PATCH /participations/{id}/hours`.
- ☐ Add `status` column to `organizations` table (`pending|verified|rejected|suspended`) — currently always `null`.
- ☐ Add `rejection_reason` + `reviewed_at` to `organizations` (or a related review-log table).
- ☐ Build `POST /organizations/{id}/verification-document` (resubmission endpoint, resets status → pending).
- ☐ Fix `OrganizationResource` to return `phone` (currently omitted despite the column existing on `users`).
- ☐ Fix `OrganizationController::update` to support logo/file uploads (currently plain field update only).
- ☐ Implement `?search=` filtering on `GET /organizations` (`index()` currently ignores it).
- ☐ Build the admin namespace: `role:admin` middleware, `GET/PATCH /admin/organizations...`.
- ☐ Build `GET /governorates` (+ admin CRUD already scaffolded client-side) — currently 100% static frontend data.
- ☐ Build `GET /notifications` (+ a mark-as-read mechanism) — currently entirely client-side/localStorage.
- ☐ Confirm/implement `POST /forgot-password`, `POST /reset-password`, `PUT /admin/profile`, `PUT /admin/password` — paths are currently frontend guesses (Fortify/Breeze convention).
- ☐ Ensure login/register responses include a `roles[]` array and a non-empty `token` string.
- ☐ Build achievement-granting logic server-side (event/observer on participation completion) per the 3 rules in §13.

Should confirm / decide (see §24 for full context):

- ☐ Confirm `citiesCoveredCount` for `GET /stats/summary`, or approve dropping it from the admin dashboard.
- ☐ Decide on a dedicated `GET /admin/dashboard` aggregate endpoint vs. keeping client-side composition.

- ☐ Decide token lifetime/expiry/refresh strategy (nothing currently modeled frontend-side).
- ☐ Decide whether re-applying after `rejected` / `withdrawn` creates a new participation row or resets the old one.
- ☐ Decide whether a 3rd gender option (`"Prefer not to say"`) should be supported.
- ☐ Decide whether achievement `a3` ("group activities") should specifically require `isGroup = true` opportunities.
- ☐ Decide the exact endpoint/collection name for opportunity cover-image uploads (not yet finalized frontend-side either).
- ☐ Confirm success-stories filtering should also require `current_volu ≥ min_volu` , not just `status = completed` .

23. Frontend ↔ Backend Contract Gaps

A. Confirmed backend gaps (from direct code/controller inspection, cited in frontend comments)

Gap	Impact	Source
<code>OpportunityController</code> — no implemented actions at all	Opportunities list/detail/create/edit/delete/status/participate all non-functional in real mode	<code>src/services/opportunities.js:418-419</code>
<code>SkillController</code> — index/store/update/destroy unimplemented; table lacks working category link	Admin skill management is mock-only	<code>src/services/skills.js:96-104</code>
<code>organizations</code> table has no <code>status</code> column	Verification state can't persist; frontend fakes <code>'pending'</code>	<code>src/utils/api/apiResponseSchemas.js:70-72</code>
<code>OrganizationResource</code> never returns <code>phone</code>	Org directory/detail pages can't show phone even though data exists	<code>src/services/organizations.js:24-28</code>
<code>OrganizationController::update</code> has no file-upload handling	Org logo can never be changed post-registration via this endpoint	<code>src/services/organization.js:1-24</code>
<code>GET /organizations</code> ignores <code>?search=</code>	Search box on org directory is currently a no-op server-side	<code>src/services/organizations.js:10-37</code>
No admin routes/ <code>role:admin</code> middleware exist	Entire admin org-review flow is mock-only	<code>src/services/admin.js:1-13</code>
No <code>hours_logged</code> column / hours endpoint	Org can't record final confirmed hours in real mode	<code>src/services/participations.js:216-222</code>
No notifications endpoint	Notification bell is fully simulated client-side	<code>src/services/notifications.js:1-13</code>

B. Frontend-internal inconsistencies (not backend's fault, but relevant context)

Inconsistency	Detail
Education-level options	Schema allows 6 values; profile form UI only offers 4
Gender options	Schema allows 2 (<code>Female</code> / <code>Male</code>); a UI component (<code>GenderBadge</code>) supports a 3rd (<code>"Prefer not to say"</code>) that can never actually be selected
Governorate lists duplicated 4×	<code>syrianGovernorates.js</code> , <code>volunteerProfile/ProfileForm.jsx</code> , <code>OrgProfile/ProfileForm.jsx</code> , and both <code>*ProfileValidation.js</code> files each hardcode their own copy, with naming drift ("Rural Damascus" vs "Rif Dimashq")
<code>CityFormModal</code> UI	Only exposes a <code>nameEn</code> input, even though the data model (and planned API payload) requires <code>nameAr</code> too — currently no way for an admin to supply it
<code>useReviewOrganizationMutation</code> response shape	Reads <code>result.status</code> / <code>result.reason</code> at the top level, while every other mutation in the app expects <code>result.data {...}</code>
<code>admin.dashboard</code> query key	Defined in <code>queryKeys.js</code> but no hook anywhere reads or writes it — either dead code or a signal of a planned-but-unbuilt endpoint
Success-stories filter	Real-mode call filters only by <code>status=completed</code> ; mock additionally requires <code>current_volu ≥ min_volu</code> — real mode can surface "failed" opportunities as success stories
Volunteer update URL vs. header comment	File header comment suggests <code>POST /api/volunteers/me</code> ; the actual implementation calls <code>POST /volunteers/{volunteerId}</code> (by id, not "me")

C. Undetermined — needs an explicit backend/product decision

1. Token expiry & refresh strategy (nothing modeled client-side).
2. Whether logout should call a server-side token-revocation endpoint (currently pure client-side clear).
3. Whether re-applying after `rejected` / `withdrawn` creates a new pivot row or resets the existing one to `pending` .
4. `citiesCoveredCount` on the platform-stats endpoint — exists in UI, unconfirmed if backend will ever supply it.
5. Whether achievements should remain platform-wide (current explicit decision) or ever become per-organization.
6. Whether achievement `a3` should require `isGroup = true` opportunities specifically.
7. Final field name / media collection for opportunity cover-image uploads.
8. Whether `pending` organizations should be allowed to post opportunities at all before verification (current mock behavior doesn't explicitly block this beyond the "verified" gate on accept/reject/hours actions).
9. Whether `GET /organizations` pagination (`paginate(15)`) needs frontend-side "load more"/page controls to be added — no such UI was found, meaning only the first page may currently ever be visible regardless of backend readiness.

End of document. All findings above trace to specific frontend source files; none were inferred beyond what the code and its inline comments state explicitly.